

Department of Computer Science and Engineering

B.Tech III Semester

Academic Year 2026-2027

Database Management Systems(2501IT05)

Capstone Project Documentation



ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh-533437

Cinema Ticket Booking System

Review-2 | 01-10-2026



SCHOOL OF COMPUTING
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Capstone project work entitled
“Cinema Ticket Booking System” is submitted by

- **M.Tejaswaoop** (25B11CS563)
- **Ch.Dilip** (25B11CS166)
- **M.Siddharth Raj** (25B11CS617)
- **M.Santosh Naik** (25B11CS614)

in partial fulfillment of the requirements for the award of the
B.Tech. degree in Computer Science and Engineering for the
academic year 2026-2027.

Project Guide

Mrs. K Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli, Ph.D,
Assistant Professor,
Department of CSE

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K Vydehi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor, Dr Hari Jyothula, Associate Professor and Dr Pennada Siva Satya Prasad, Assistant Professor**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

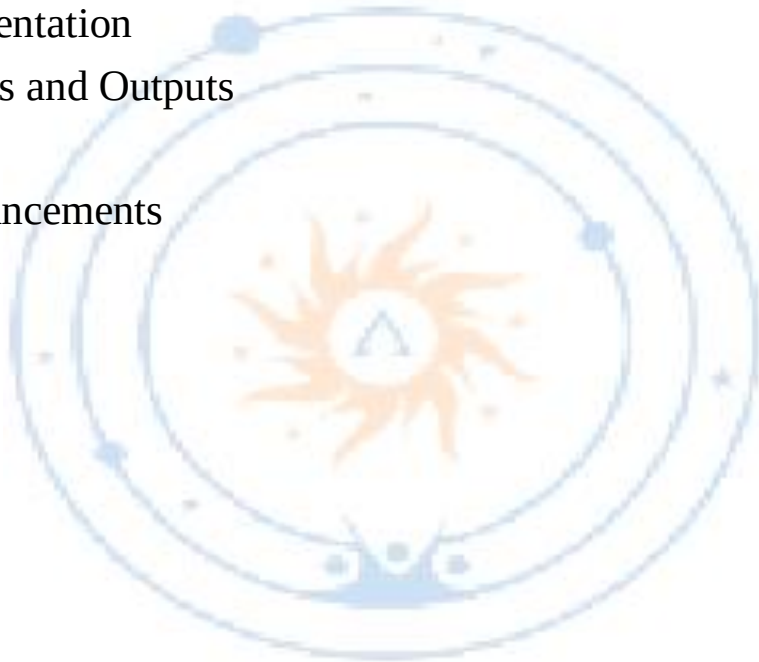
We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

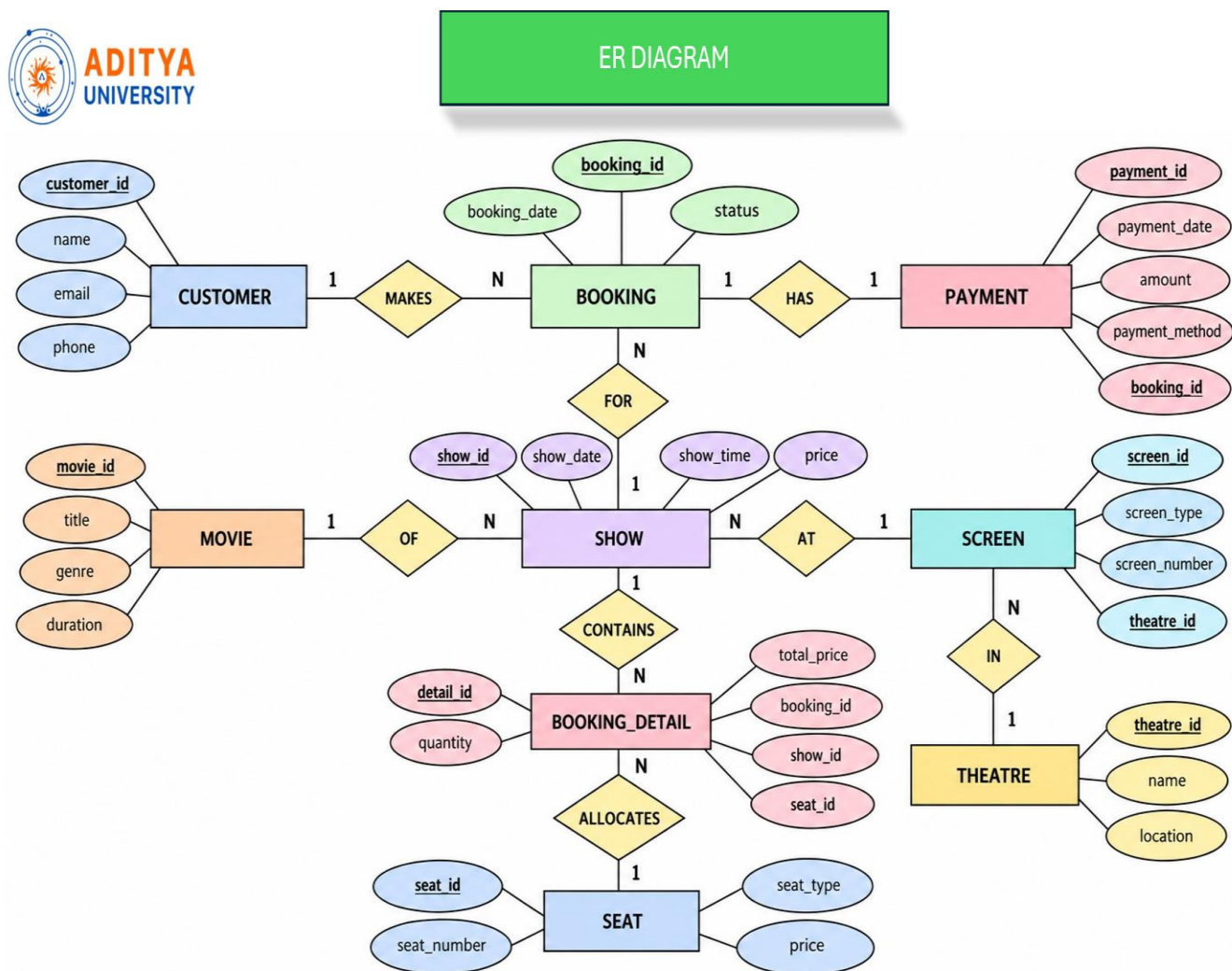
TABLE OF CONTENTS

1. Abstract
2. Introduction
3. System Analysis
4. Database Design
5. ER Diagram
6. Relational Schema
7. Table Design
8. Normalization
9. SQL Implementation
10. SQL Queries and Outputs
11. Conclusion
12. Future Enhancements
13. References



LIST OF FIGURE

Figure 1. ER representation of the Cinema Ticket Booking System



LIST OF TABLES

Table 1. Major functional modules and requirements

Table 2. Major entities and their purposes

Table 3. Relationships and cardinalities

Table 4. Relational schema

Table 5. CUSTOMER table structure

Table 6. BOOKING table structure

Table 7. PAYMENT table structure

Table 8. MOVIE table structure

Table 9. SHOW table structure

Table 10. SCREEN table structure

Table 11. THEATRE table structure

Table 12. BOOKING_DETAIL table structure

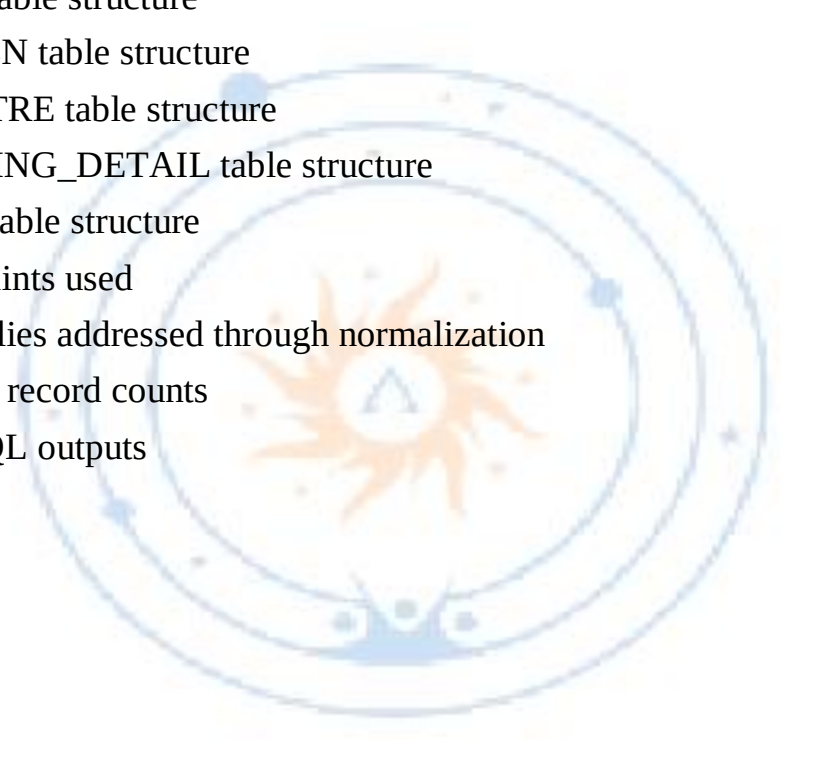
Table 13. SEAT table structure

Table 14. Constraints used

Table 15. Anomalies addressed through normalization

Table 16. Sample record counts

Table 17. Key SQL outputs



ACRONYMS

DBMS — Database Management System

ER — Entity Relationship

SQL — Structured Query Language

DDL — Data Definition Language

DML — Data Manipulation Language

DQL — Data Query Language

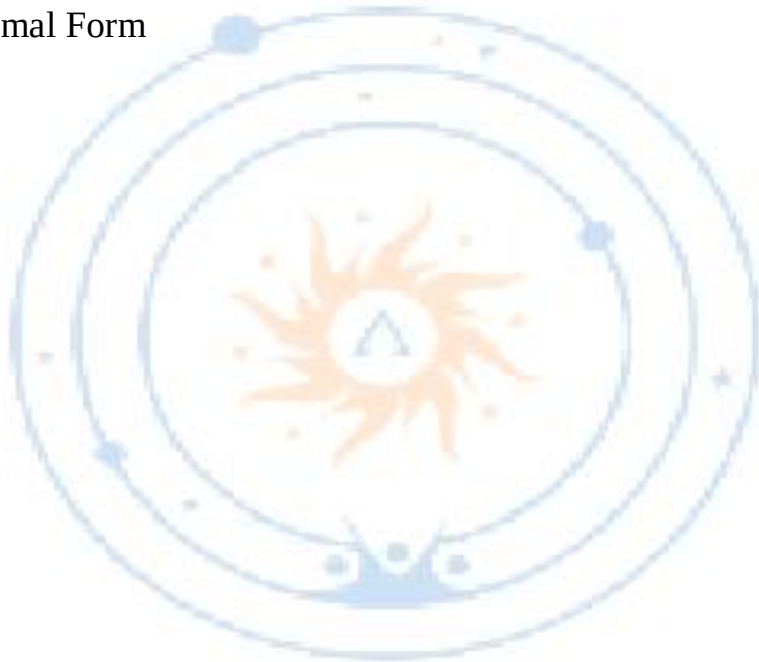
PK — Primary Key

FK — Foreign Key

1NF — First Normal Form

2NF — Second Normal Form

3NF — Third Normal Form



1. ABSTRACT

The Cinema Ticket Booking System is a centralized relational database designed to manage customers, movies, shows, theatres, screens, seats, bookings, booking details, and payments. The supplied project identifies common problems in manual cinema ticket operations, including long queues, difficulty checking seat availability, incorrect seat allocation, duplicate reservations, manual show management, scattered payment records, and complicated cancellation tracking. The proposed database organizes these activities using related tables connected through primary and foreign keys.

The implementation follows a structured relational design and demonstrates SQL operations for data definition, insertion, filtering, ordering, joins, grouping, aggregation, subqueries, and existence checks. The project contains nine entities and documents normalization through 1NF, 2NF, and 3NF to reduce redundancy and prevent insertion, update, and deletion anomalies. The query set reports customer bookings, movie duration, show pricing, payment methods, theatre and screen information, unpaid customers, theatres without shows, and total booking amounts, providing a practical DBMS representation of cinema operations.

2. INTRODUCTION

2.1 Project Overview

The Cinema Ticket Booking System provides a structured database for managing ticket-booking activities in a cinema environment. It maintains customer details, movie information, show schedules, theatre and screen details, seat information, bookings, booking details, and payment records. A centralized database enables these related records to be stored consistently and retrieved through SQL queries instead of relying on separate manual records.

2.2 Problem Statement

- Long queues: Manual ticket booking can require customers to wait at the theatre counter.
- Seat availability issues: Customers may not have a convenient way to know which seats are available before booking.
- Booking errors: Manual booking can lead to incorrect seat allocation or duplicate reservations.
- Difficulty managing shows: Managing movies, show timings, screens, and seat availability manually can be time-consuming.
- Payment and booking records: Maintaining customer booking and payment records manually can be difficult.
- Cancellation management: Tracking cancelled tickets and updating seat availability can become complicated.
- Data management: A centralized database makes it easier to store, update, search, and retrieve cinema and booking information.

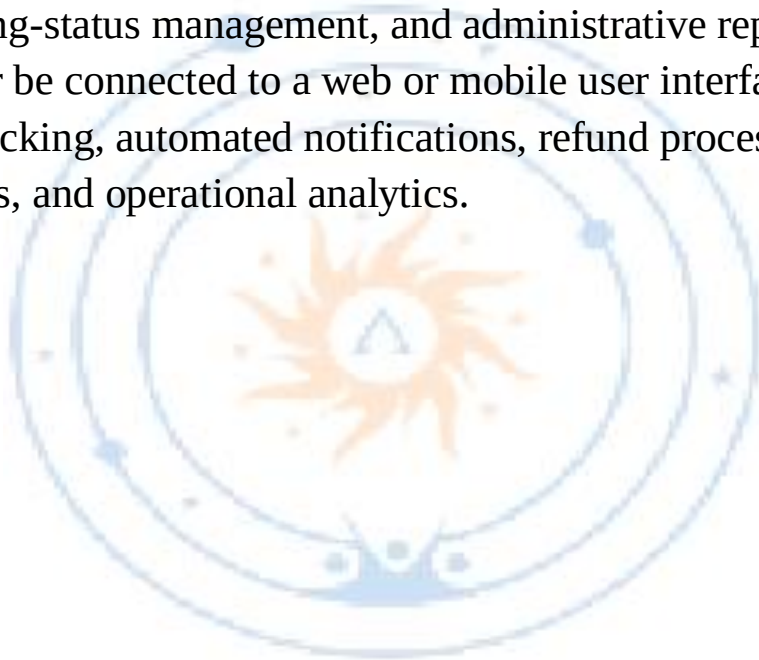
2.3 Objectives

- Develop a simple and user-friendly cinema ticket booking system.
- Maintain customer details efficiently using a database.
- Manage movie information and available show timings.
- Provide easy seat selection and booking facilities.

- Maintain accurate booking and payment records.
- Reduce errors and duplicate seat reservations.
- Allow administrators to manage movies, shows, theatres, and bookings.
- Use SQL database operations for efficient storage, updating, searching, and retrieval of cinema-related information.

2.4 Scope

The project scope covers customer, movie, show, theatre, screen, seat, booking, booking-detail, and payment management. The database provides the foundation for ticket search, seat selection, reservation recording, payment tracking, booking-status management, and administrative reporting. The system can later be connected to a web or mobile user interface and extended with live seat locking, automated notifications, refund processing, richer security controls, and operational analytics.



3. SYSTEM ANALYSIS

3.1 Existing System

- Manual record maintenance for customers, shows, seats, and bookings.
- Customers may need to depend on counter staff for current seat availability.
- Repeated customer and booking information increases the possibility of inconsistent records.
- Manual handling can produce duplicate or incorrect seat allocations.
- Payment and cancellation information may be scattered across separate records.
- Generating consolidated reports for shows, theatres, payments, and booking amounts is time-consuming.

3.2 Proposed System

The proposed system uses a centralized relational database in which each major real-world object is represented by a table. Customer, booking, payment, movie, show, screen, theatre, booking-detail, and seat information are stored separately and linked through primary and foreign keys. This structure makes it possible to retrieve related information through JOIN operations while maintaining clear ownership of each data item.

3.3 Functional Requirements

Module	Functional Requirement
User Registration/Login	Manage customer accounts and identify customers for booking activities.
Movie & Show Management	Add, update, and view movies and show timings.
Seat Booking	View available seats and record seat selections for a show.
Payment Management	Record ticket payments and payment methods associated with bookings.
Booking Management	Create, view, and cancel bookings while maintaining booking status.
Theatre & Screen Management	Maintain theatres, screen numbers, screen types, and their relationships.
Reporting	Use SQL queries to generate booking, payment, show, theatre, and customer reports.

3.4 Non-Functional Requirements

- Usability: The database-backed system should support simple and user-friendly booking operations.
- Security: Customer and booking information should be protected through controlled access and appropriate authorization.
- Reliability: Booking records should be maintained accurately and should prevent duplicate or incorrect reservations.
- Performance: Customer, movie, show, theatre, seat, and booking queries should return records efficiently.
- Data Integrity: Primary-key and foreign-key constraints should maintain valid relationships among the nine tables.
- Scalability: The design should allow the number of customers, shows, seats, and bookings to grow without changing the core relational structure.

3.5 End Users

User	Main Interaction
Customers	Search movies, select seats, make bookings, review booking status, and make payments.
Administrator	Manage movies, shows, seats, theatres, screens, customers, bookings, and payments.

3.6 Development Methodology

- Requirement analysis identifies the operational problems, system objectives, user roles, and required functions.
- Conceptual design represents cinema objects as entities and defines their relationships and cardinalities.
- Relational mapping converts the ER model into tables with primary and foreign keys.
- Normalization organizes the relations through 1NF, 2NF, and 3NF to reduce redundancy and anomalies.

- SQL implementation creates the tables, loads sample data, and demonstrates retrieval through 15 queries.
- Validation compares the displayed query results with the sample records and the relationships shown in the supplied presentation.



4. DATABASE DESIGN

4.1 Major Entities

Entity	Purpose
CUSTOMER	Stores customer identity and contact information.
BOOKING	Stores each customer booking, booking date, status, and customer reference.
PAYMENT	Stores payment date, amount, payment method, and booking reference.
MOVIE	Stores movie title, genre, and duration.
SHOW	Stores show date, show time, price, and references to the movie and screen.
SCREEN	Stores screen type, screen number, and theatre reference.
THEATRE	Stores theatre name and location.
BOOKING_DETAIL	Stores quantity, total price, and references to booking, show, and seat.
SEAT	Stores seat number, price, and seat type.



4.2 Relationships and Cardinality

Relationship	Cardinality	Description
CUSTOMER → BOOKING	1 : N	One customer can make many bookings; each booking belongs to one customer.
BOOKING → PAYMENT	1 : 1	One booking has one payment; each payment is associated with one booking.
BOOKING → SHOW	N : 1	Many bookings can be associated with the same show; booking details identify the show involved.
MOVIE → SHOW	1 : N	One movie can have many shows; each show is for one movie.
SHOW → SCREEN	N : 1	Many shows can occur on the same screen; each show takes place on one screen.
SCREEN → THEATRE	N : 1	A theatre can have many screens; each screen belongs to one theatre.
SHOW → BOOKING_DETAIL	1 : N	One show can have many booking details; each booking detail belongs to one show.
BOOKING_DETAIL → SEAT	N : 1	Many booking details can refer to a seat at different shows; each booking detail refers to one seat.

5. ER DIAGRAM

The following diagram is the documentation representation derived directly from the nine entities, foreign keys, and relationship cardinalities presented in the supplied cinema-ticket-booking project.

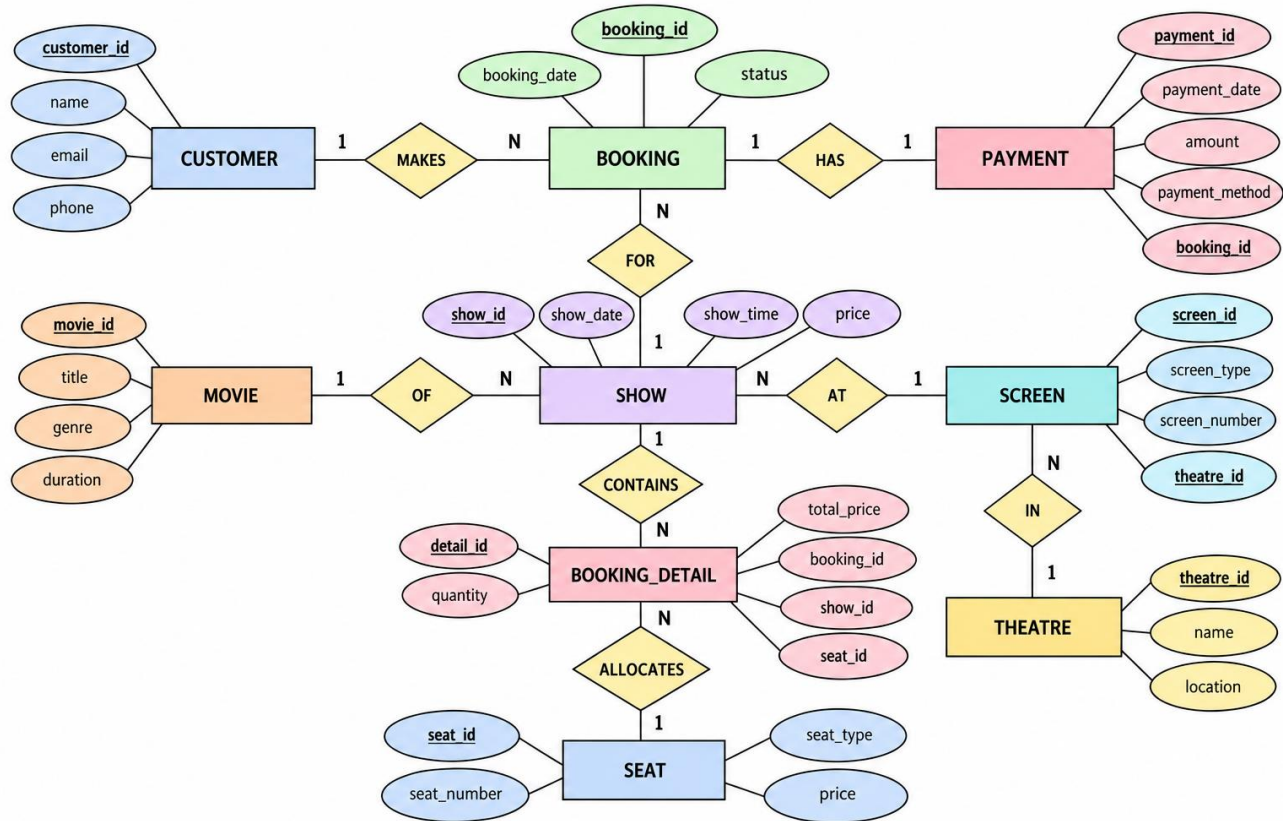


Figure 1. ER representation of the Cinema Ticket Booking System

The ER model shows CUSTOMER making BOOKING records; BOOKING is associated with PAYMENT and SHOW activity; MOVIE defines SHOW content; SHOW occurs on a SCREEN; SCREEN belongs to a THEATRE; and BOOKING_DETAIL connects bookings and shows to individual SEAT records. Primary-key attributes are underlined in the supplied diagram, while foreign-key references connect related entities.

6. RELATIONAL SCHEMA

Relation	Relational Schema
CUSTOMER	customer_id (PK), name, email, phone
BOOKING	booking_id (PK), booking_date, status, customer_id (FK)
PAYMENT	payment_id (PK), payment_date, amount, payment_method, booking_id (FK)
MOVIE	movie_id (PK), title, genre, duration
SHOW	show_id (PK), show_date, show_time, price, movie_id (FK), screen_id (FK)
SCREEN	screen_id (PK), screen_type, screen_number, theatre_id (FK)
THEATRE	theatre_id (PK), name, location
BOOKING_DETAIL	detail_id (PK), quantity, total_price, booking_id (FK), show_id (FK), seat_id (FK)
SEAT	seat_id (PK), seat_number, price, seat_type

PK = Primary Key; FK = Foreign Key. The relational schema preserves the entity attributes stated in the supplied presentation and carries relationship references through foreign keys.

7. TABLE DESIGN

7.1 CUSTOMER

Attribute	Data Type	Constraint	Description
customer_id	INT	PK	Unique customer identifier
name	VARCHAR2(30)	—	Customer name
email	VARCHAR2(30)	—	Customer email address
phone	VARCHAR2(15)	—	Customer contact number

7.2 BOOKING

Attribute	Data Type	Constraint	Description
booking_id	INT	PK	Unique booking identifier
booking_date	DATE	—	Date on which the booking is recorded
status	VARCHAR2(30)	—	Booking status such as CONFIRMED, PENDING, or CANCELLED
customer_id	INT	FK	References CUSTOMER(customer_id)

7.3 PAYMENT

Attribute	Data Type	Constraint	Description
payment_id	INT	PK	Unique payment identifier
payment_date	DATE	—	Date on which payment is recorded
amount	INT	—	Payment amount
payment_method	VARCHAR2(30)	—	Method used for payment
booking_id	INT	FK	References BOOKING(booking_id)

7.4 MOVIE

Attribute	Data Type	Constraint	Description
movie_id	INT	PK	Unique movie identifier
title	VARCHAR2(30)	—	Movie title
genre	VARCHAR2(30)	—	Movie genre
duration	INT	—	Movie duration in minutes

7.5 SHOW

Attribute	Data Type	Constraint	Description
show_id	INT	PK	Unique show identifier
show_time	TIMESTAMP	—	Scheduled show date and time
price	INT	—	Ticket price for the show
show_date	DATE	—	Date of the show
movie_id	INT	FK	References MOVIE(movie_id)
screen_id	INT	FK	References SCREEN(screen_id)

7.6 SCREEN

Attribute	Data Type	Constraint	Description
screen_id	INT	PK	Unique screen identifier
screen_type	VARCHAR2(30)	—	Screen type such as IMAX, 2D, or 3D
screen_number	INT	—	Screen number within the theatre
theatre_id	INT	FK	References THEATRE(theatre_id)

7.7 THEATRE

Attribute	Data Type	Constraint	Description
theatre_id	INT	PK	Unique theatre identifier
name	VARCHAR2(40)	—	Theatre name
location	VARCHAR2(40)	—	Theatre location

7.8 BOOKING_DETAIL

Attribute	Data Type	Constraint	Description
detail_id	INT	PK	Unique booking-detail identifier
quantity	INT	—	Number of seats/items represented by the detail
total_price	INT	—	Total price for the booking detail
booking_id	INT	FK	References BOOKING(booking_id)
show_id	INT	FK	References SHOW(show_id)
seat_id	INT	FK	References SEAT(seat_id)

7.9 SEAT

Attribute	Data Type	Constraint	Description
seat_id	INT	PK	Unique seat identifier
seat_number	INT	—	Seat number within a screen
price	INT	—	Base seat price
seat_type	VARCHAR2(30)	—	Seat type such as REGULAR, PREMIUM, or RECLINER

7.10 Constraints Used

Constraint	Where Used	Purpose
PRIMARY KEY (PK)	All 9 tables	Uniquely identifies each record.
FOREIGN KEY (FK)	SCREEN, SEAT, SHOW, BOOKING, BOOKING_DETAIL, PAYMENT	Connects related tables and maintains referential relationships.
NOT NULL	Not explicitly used in the supplied code	Prevents a column from having NULL values.
UNIQUE	Not explicitly used	Prevents duplicate values when defined.
CHECK	Not explicitly used	Restricts values according to a defined condition.

8. NORMALIZATION

Normalization is applied step-by-step to organize the cinema database, reduce data redundancy, and prevent insertion, update, and deletion anomalies. The presentation demonstrates the process using 1NF, 2NF, and 3NF with examples involving CUSTOMER and BOOKING relations.

8.1 Unnormalized Form (UNF)

An unnormalized cinema booking record can combine customer, booking, payment, movie, show, screen, theatre, booking-detail, and seat facts in one large record. Such a flat structure repeats customer and booking information whenever the same customer makes more than one booking and mixes facts that have different dependencies.

```
CINEMA_BOOKING_DATA(  
  customer_id, name, email, phone,  
  booking_id, booking_date, status,  
  payment_id, payment_date, amount, payment_method,  
  movie_id, title, genre, duration,  
  show_id, show_date, show_time, price,  
  screen_id, screen_type, screen_number,  
  theatre_id, theatre_name, location,  
  detail_id, quantity, total_price,  
  seat_id, seat_number, seat_price, seat_type  
)
```

8.2 First Normal Form (1NF)

- All attributes contain atomic (single) values.
- There are no repeating groups or multi-valued attributes.
- Each record can be identified by a primary key.
- Separate relations are used for entities such as CUSTOMER, MOVIE, BOOKING, and SEAT.
- Appropriate data types are assigned to each column.

In the supplied presentation, CUSTOMER and MOVIE are shown as 1NF relations with atomic attributes. CUSTOMER stores one customer per row, while MOVIE stores one movie per row with a single title, genre, and duration value.

CUSTOMER_ID	NAME	EMAIL	PHONE
1	Rahul	rahul@gmail.com	9876543210
2	Priya	priya@gmail.com	9876543211
3	Arjun	arjun@gmail.com	9876543212
4	Sneha	sneha@gmail.com	9876543213
5	Kiran	kiran@gmail.com	9876543214

MOVIE_ID	TITLE	GENRE	DURATION
301	Interstellar	Sci-Fi	169
302	Inception	Sci-Fi	148
303	Avengers	Action	143
304	Joker	Drama	122

8.3 Second Normal Form (2NF)

The supplied normalization example shows a combined 1NF relation containing BOOKING and CUSTOMER information. Customer attributes are stored separately in CUSTOMER, while BOOKING retains customer_id as a foreign key. This removes the repeated customer details from every booking record and produces a cleaner dependency structure.

BOOKING_ID	BOOKING_DATE	STATUS	CUSTOMER_ID
501	09-SEP-26	CONFIRMED	1
502	09-SEP-26	CONFIRMED	2
503	09-SEP-26	CANCELLED	3
504	09-SEP-26	CONFIRMED	4

8.4 Third Normal Form (3NF)

The 3NF design stores each non-key fact with the entity whose primary key determines that fact. CUSTOMER stores name, email, and phone under customer_id; MOVIE stores title, genre, and duration under movie_id; THEATRE stores its name and location under theatre_id; and BOOKING references the customer through customer_id rather than repeating customer

attributes. The same principle is applied to SHOW, SCREEN, SEAT, PAYMENT, and BOOKING_DETAIL.

```
CUSTOMER(customer_id, name, email, phone)
BOOKING(booking_id, booking_date, status, customer_id)
MOVIE(movie_id, title, genre, duration)
SHOW(show_id, show_date, show_time, price, movie_id, screen_id)
SCREEN(screen_id, screen_type, screen_number, theatre_id)
THEATRE(theatre_id, name, location)
SEAT(seat_id, seat_number, price, seat_type)
PAYMENT(payment_id, payment_date, amount, payment_method, booking_id)
BOOKING_DETAIL(detail_id, quantity, total_price, booking_id, show_id, seat_id)
```

8.5 Anomalies Removed

Anomaly	Before Normalization	After 3NF
Insertion	Customer information cannot be added without booking-related data in an unnormalized table.	Normalization separates CUSTOMER and BOOKING, allowing a new customer to be inserted independently.
Update	Customer details may be repeated across multiple booking records. Updating one customer may require multiple row changes.	Customer details are stored once in CUSTOMER and referenced from BOOKING.
Deletion	Deleting a booking may also remove customer information in an unnormalized table.	A booking can be deleted without losing customer details because the relations are separated.
Redundancy	Movie, theatre, customer, screen, and seat facts can be repeated across flat booking records.	Information is stored once in its own entity relation and connected through keys.

Normalization therefore produces a well-structured, consistent, and maintainable database while keeping the relationships needed for ticket booking through primary and foreign keys.

9. SQL IMPLEMENTATION

The supplied presentation contains Oracle-style CREATE TABLE and INSERT examples for the cinema database. The following code blocks document the nine-table implementation using the attributes stated in the presentation. The PAYMENT table definition is included from the stated nine-entity schema because a complete row-level PAYMENT creation screenshot is not visible in the supplied presentation; the presentation does, however, use PAYMENT in multiple SQL queries.

9.1 DDL — Core Entity Tables

```
CREATE TABLE CUSTOMER (  
  customer_id INT PRIMARY KEY,  
  name VARCHAR2(30),  
  email VARCHAR2(30),  
  phone VARCHAR2(15)  
);  
  
CREATE TABLE THEATRE (  
  theatre_id INT PRIMARY KEY,  
  name VARCHAR2(40),  
  location VARCHAR2(40)  
);  
  
CREATE TABLE MOVIE (  
  movie_id INT PRIMARY KEY,  
  title VARCHAR2(30),  
  genre VARCHAR2(30),  
  duration INT  
);
```

9.2 DDL — Relationship-Dependent Tables

```
CREATE TABLE SCREEN (  
  screen_id INT PRIMARY KEY,  
  screen_type VARCHAR2(30),  
  screen_number INT,  
  theatre_id INT,  
  FOREIGN KEY (theatre_id) REFERENCES THEATRE(theatre_id)  
);  
  
CREATE TABLE SEAT (  
  seat_id INT PRIMARY KEY,  
  seat_number INT,  
  price INT,  
  seat_type VARCHAR2(30),  
  screen_id INT,  
  FOREIGN KEY (screen_id) REFERENCES SCREEN(screen_id)  
);
```

```

CREATE TABLE BOOKING (
    booking_id INT PRIMARY KEY,
    booking_date DATE,
    status VARCHAR2(30),
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id)
);

CREATE TABLE "SHOW" (
    show_id INT PRIMARY KEY,
    show_time TIMESTAMP,
    price INT,
    show_date DATE,
    movie_id INT,
    screen_id INT,
    FOREIGN KEY (movie_id) REFERENCES MOVIE(movie_id),
    FOREIGN KEY (screen_id) REFERENCES SCREEN(screen_id)
);

CREATE TABLE PAYMENT (
    payment_id INT PRIMARY KEY,
    payment_date DATE,
    amount INT,
    payment_method VARCHAR2(30),
    booking_id INT,
    FOREIGN KEY (booking_id) REFERENCES BOOKING(booking_id)
);

CREATE TABLE BOOKING_DETAIL (
    detail_id INT PRIMARY KEY,
    quantity INT,
    total_price INT,
    booking_id INT,
    show_id INT,
    seat_id INT,
    FOREIGN KEY (booking_id) REFERENCES BOOKING(booking_id),
    FOREIGN KEY (show_id) REFERENCES "SHOW"(show_id),
    FOREIGN KEY (seat_id) REFERENCES SEAT(seat_id)
);

```

9.3 Sample Data

The supplied presentation displays sample rows for CUSTOMER, THEATRE, SCREEN, SEAT, MOVIE, SHOW, BOOKING, and BOOKING_DETAIL. It also uses payment data through the aggregate query output. The rows below reproduce the visible sample data needed for the documented queries.

CUSTOMER_ID	NAME	EMAIL	PHONE
1	Rahul	rahul@gmail.com	9876543210
2	Priya	priya@gmail.com	9876543211
3	Arjun	arjun@gmail.com	9876543212
4	Sneha	sneha@gmail.com	9876543213
5	Kiran	kiran@gmail.com	9876543214
6	Ananya	ananya@gmail.com	9876543215
7	Rohit	rohit@gmail.com	9876543216
8	Meera	meera@gmail.com	9876543217
9	Vikram	vikram@gmail.com	9876543218
10	Divya	divya@gmail.com	9876543219

THEATRE_ID	NAME	LOCATION
101	PVR Cinemas	Hyderabad
102	INOX	Bangalore
103	Cinepolis	Chennai
104	Miraj Cinemas	Mumbai
105	Asian Cinemas	Vizag

SCREEN_ID	SCREEN_TYPE	SCREEN_NUMBER	THEATRE_ID
201	IMAX	1	101
202	2D	2	101
203	3D	1	102
204	2D	2	102
205	IMAX	1	103
206	2D	2	103
207	3D	1	104
208	2D	2	104
209	IMAX	1	105
210	2D	2	105

MOVIE_ID	TITLE	GENRE	DURATION
301	Interstellar	Sci-Fi	169
302	Inception	Sci-Fi	148
303	Avengers	Action	143
304	Joker	Drama	122

SHOW_ID	SHOW_TIME	PRICE	SHOW_DATE	MOVIE_ID	SCREEN_ID
401	2026-09-10 10:00:00	250	2026-09-10	301	201
402	2026-09-10 14:00:00	200	2026-09-10	302	203
403	2026-09-10 18:00:00	300	2026-09-10	303	205
404	2026-09-10 21:00:00	220	2026-09-10	304	207

BOOKING_ID	BOOKING_DATE	STATUS	CUSTOMER_ID
501	2026-09-09	CONFIRMED	1
502	2026-09-09	CONFIRMED	2
503	2026-09-09	CANCELLED	3
504	2026-09-09	CONFIRMED	4
505	2026-09-09	PENDING	5
506	2026-09-09	CONFIRMED	6
507	2026-09-09	CONFIRMED	7
508	2026-09-09	CANCELLED	8
509	2026-09-09	CONFIRMED	9
510	2026-09-09	CONFIRMED	10

DETAIL_ID	QUANTITY	TOTAL_PRICE	BOOKING_ID	SHOW_ID	SEAT_ID
701	1	250	501	401	601
702	1	200	502	402	605
703	1	300	503	403	609
704	1	220	504	404	613
705	1	250	505	401	602
706	1	200	506	402	606
707	1	300	507	403	610
708	1	220	508	404	614
709	1	250	509	401	603
710	1	200	510	402	621

SEAT_ID	SEAT_NUMBER	PRICE	SEAT_TYPE	SCREEN_ID
601	1	250	PREMIUM	201
602	2	200	REGULAR	201
603	1	250	PREMIUM	202
604	2	200	REGULAR	202
605	1	300	RECLINER	203

SEAT_ID	SEAT_NUMBER	PRICE	SEAT_TYPE	SCREEN_ID
606	2	250	PREMIUM	203
607	1	250	PREMIUM	204
608	2	200	REGULAR	204
609	1	300	RECLINER	205
610	2	250	PREMIUM	205
611	1	250	PREMIUM	206
612	2	200	REGULAR	206
613	1	300	RECLINER	207
614	2	250	PREMIUM	207
615	1	250	PREMIUM	208
616	2	200	REGULAR	208
617	1	300	RECLINER	209
618	2	250	PREMIUM	209
619	1	250	PREMIUM	210
620	2	200	REGULAR	210
621	3	250	PREMIUM	203
622	4	200	REGULAR	203

9.4 Sample Record Counts

Table	Records in supplied project
CUSTOMER	10
THEATRE	5
SCREEN	10
SEAT	22
MOVIE	4 visible rows in the table-creation snapshot
SHOW	4
BOOKING	10
BOOKING_DETAIL	10
PAYMENT	Row-level count not explicitly displayed in the presentation

Source consistency note: the presentation contains more than one theatre snapshot. The documented sample uses theatre IDs 101–105 because those values are the ones referenced by the SCREEN records shown in the same presentation. Query outputs are reproduced as displayed even where a query

snapshot shows additional movie rows not visible in the table-creation snapshot.



10. SQL QUERIES AND OUTPUTS

The supplied presentation demonstrates 15 SQL queries, beginning with basic single-table selection and continuing through filtering, ordering, aggregate functions, joins, subqueries, NOT EXISTS, LEFT JOIN, GROUP BY, and NVL. Each query below includes its purpose or concept followed by the displayed output.

Q1. Display the details of all customers.

Purpose / concept: Simple projection of a single table; confirms the CUSTOMER records are available.

```
SELECT *  
FROM CUSTOMER;
```

CUSTOMER_ID	NAME	EMAIL	PHONE
1	Rahul	rahul@gmail.com	9876543210
2	Priya	priya@gmail.com	9876543211
3	Arjun	arjun@gmail.com	9876543212
4	Sneha	sneha@gmail.com	9876543213
5	Kiran	kiran@gmail.com	9876543214
6	Ananya	ananya@gmail.com	9876543215
7	Rohit	rohit@gmail.com	9876543216
8	Meera	meera@gmail.com	9876543217
9	Vikram	vikram@gmail.com	9876543218
10	Divya	divya@gmail.com	9876543219

10 rows selected.

Q2. Display movies with a duration greater than 140 minutes.

Purpose / concept: WHERE filters movies using a numeric condition on duration.

```
SELECT title, genre, duration  
FROM MOVIE  
WHERE duration > 140;
```

TITLE	GENRE	DURATION
Interstellar	Sci-Fi	169
Inception	Sci-Fi	148
Oppenheimer	Biography	180
The Dark Knight	Action	152

4 rows selected.

Q3. Display all customers in alphabetical order.

Purpose / concept: ORDER BY sorts the customer names in ascending alphabetic order.

```
SELECT customer_id, name, email
FROM CUSTOMER
ORDER BY name;
```

CUSTOMER_ID	NAME	EMAIL
6	Ananya	ananya@gmail.com
3	Arjun	arjun@gmail.com
10	Divya	divya@gmail.com
5	Kiran	kiran@gmail.com
8	Meera	meera@gmail.com
2	Priya	priya@gmail.com
1	Rahul	rahul@gmail.com
7	Rohit	rohit@gmail.com
4	Sneha	sneha@gmail.com
9	Vikram	vikram@gmail.com

10 rows selected.

Q4. Find the highest show price.

Purpose / concept: MAX() returns the largest ticket price stored in the SHOW relation.

```
SELECT MAX(price) AS highest_price
FROM "SHOW";
```

HIGHEST_PRICE
300

1 row selected.

Q5. Find the average duration of all movies.

Purpose / concept: AVG() calculates the mean movie duration; ROUND() displays two decimal places.

```
SELECT ROUND(AVG(duration), 2) AS average_duration  
FROM MOVIE;
```

AVERAGE_DURATION
145.50

1 row selected.

Q6. Display all confirmed bookings.

Purpose / concept: WHERE filters BOOKING records whose status is CONFIRMED.

```
SELECT booking_id, booking_date, customer_id  
FROM BOOKING  
WHERE status = 'CONFIRMED';
```

BOOKING_ID	BOOKING_DATE	CUSTOMER_ID
501	09-SEP-26	1
502	09-SEP-26	2
504	09-SEP-26	4
506	09-SEP-26	6
507	09-SEP-26	7
509	09-SEP-26	9
510	09-SEP-26	10

7 rows selected.

Q7. Display different payment methods.

Purpose / concept: DISTINCT returns each payment method only once.

```
SELECT DISTINCT payment_method  
FROM PAYMENT;
```

PAYMENT_METHOD
UPI
CREDIT CARD
CASH
DEBIT CARD
NET BANKING

5 rows selected.

Q8. Display customer names with their booking details.

Purpose / concept: INNER JOIN combines CUSTOMER and BOOKING through customer_id and sorts by booking_id.

```
SELECT c.name, b.booking_id, b.status
FROM CUSTOMER c
JOIN BOOKING b
ON c.customer_id = b.customer_id
ORDER BY b.booking_id;
```

NAME	BOOKING_ID	STATUS
Rahul	501	CONFIRMED
Priya	502	CONFIRMED
Arjun	503	CANCELLED
Sneha	504	CONFIRMED
Kiran	505	PENDING
Ananya	506	CONFIRMED
Rohit	507	CONFIRMED
Meera	508	CANCELLED
Vikram	509	CONFIRMED
Divya	510	CONFIRMED

10 rows selected.

Q9. Display movie, show, screen and theatre details.

Purpose / concept: Multiple JOIN operations traverse MOVIE → SHOW → SCREEN → THEATRE to produce a consolidated schedule report.

```
SELECT m.title, s.show_id, s.show_time, sc.screen_number,
       t.name AS theatre_name
FROM MOVIE m
JOIN "SHOW" s ON m.movie_id = s.movie_id
JOIN SCREEN sc ON s.screen_id = sc.screen_id
JOIN THEATRE t ON sc.theatre_id = t.theatre_id
ORDER BY s.show_id;
```

TITLE	SHOW_ID	SHOW_TIME	SCREEN_NUMBER	THEATRE_NAME
Interstellar	401	10:00	1	PVR Cinemas
Inception	402	14:00	1	INOX
Avengers	403	18:00	1	Cinepolis
Joker	404	21:00	1	Miraj Cinemas

4 rows selected.

Q10. Count the number of screens in each theatre.

Purpose / concept: JOIN combines theatre and screen records; COUNT() and GROUP BY aggregate screen totals per theatre.

```
SELECT t.name AS theatre_name, COUNT(sc.screen_id) AS total_screens
FROM THEATRE t
JOIN SCREEN sc ON t.theatre_id = sc.theatre_id
GROUP BY t.name
ORDER BY t.name;
```

THEATRE_NAME	TOTAL_SCREEN
Asian Cinemas	2
Cinepolis	2
INOX	2
Miraj Cinemas	2
PVR Cinemas	2

5 rows selected.

Q11. Calculate total payment amount for each payment method.

Purpose / concept: SUM() with GROUP BY calculates payment totals for every payment method.

```
SELECT payment_method, SUM(amount) AS total_amount
FROM PAYMENT
GROUP BY payment_method
ORDER BY total_amount DESC;
```

PAYMENT_METHOD	TOTAL_AMOUNT
UPI	750
NET BANKING	250
CASH	220
CREDIT CARD	200
DEBIT CARD	200

5 rows selected.

Q12. Display movies whose duration is greater than the average movie duration.

Purpose / concept: A subquery computes AVG(duration) and the outer query selects movies above that average.

```
SELECT title, duration
FROM MOVIE
WHERE duration > (
    SELECT AVG(duration)
    FROM MOVIE
);
```

TITLE	DURATION
Interstellar	169
Inception	148

2 rows selected.

Q13. Find customers who have not made a payment.

Purpose / concept: NOT EXISTS identifies customers for whom no booking with a related PAYMENT record exists.

```
SELECT c.customer_id, c.name
FROM CUSTOMER c
WHERE NOT EXISTS (
    SELECT 1
    FROM BOOKING b
    JOIN PAYMENT p ON b.booking_id = p.booking_id
    WHERE b.customer_id = c.customer_id
);
```

CUSTOMER_ID	NAME
3	Arjun
5	Kiran
8	Meera

3 rows selected.

Q14. Find theatres that currently have no shows.

Purpose / concept: LEFT JOIN preserves theatres without matching screens/shows; IS NULL filters theatres with no show record.

```
SELECT t.theatre_id, t.name AS theatre_name
FROM THEATRE t
LEFT JOIN SCREEN sc ON t.theatre_id = sc.theatre_id
LEFT JOIN "SHOW" s ON sc.screen_id = s.screen_id
WHERE s.show_id IS NULL;
```

THEATRE_ID	THEATRE_NAME
105	Asian Cinemas

1 row selected.

Q15. Display the total booking amount for each customer.

Purpose / concept: LEFT JOIN retains customers even without bookings; NVL(SUM(...), 0) produces a zero total when no booking detail exists.

```
SELECT c.customer_id, c.name, NVL(SUM(bd.total_price), 0) AS total_booking_amount
FROM CUSTOMER c
LEFT JOIN BOOKING b ON c.customer_id = b.customer_id
LEFT JOIN BOOKING_DETAIL bd ON b.booking_id = bd.booking_id
GROUP BY c.customer_id, c.name
ORDER BY total_booking_amount DESC;
```

CUSTOMER_ID	NAME	TOTAL_BOOKING_AMOUNT
1	Rahul	250
2	Priya	200
3	Arjun	300
4	Sneha	220
5	Kiran	250
6	Ananya	200
7	Rohit	300
8	Meera	220
9	Vikram	250
10	Divya	200

10 rows selected.

10.1 Key Outputs from the Supplied Data

Query / Measure	Supplied Result
Highest show price	300
Average movie duration	145.50 minutes
Confirmed bookings	7
Distinct payment methods	5
Customers without a payment	Arjun (3), Kiran (5), Meera (8)
Theatre with no shows	105 — Asian Cinemas
Screens per theatre in the shown result	2 each

Source note: Query snapshots reproduce the presentation output, including additional movie rows not visible in the separate table-creation snapshot.

11. CONCLUSION

The Cinema Ticket Booking System converts common cinema-ticket activities into a structured relational database. The design separates customer, booking, payment, movie, show, screen, theatre, booking-detail, and seat information and connects them through primary and foreign keys. This makes the database easier to maintain and supports consistent retrieval of related cinema data.

The SQL implementation demonstrates practical DBMS concepts including DDL, table creation, foreign-key relationships, INSERT-based sample loading, SELECT statements, WHERE filters, ORDER BY sorting, JOIN operations, GROUP BY, aggregate functions, subqueries, NOT EXISTS, LEFT JOIN, DISTINCT, and NVL. The 15 queries provide reports for customers, movies, show prices, booking status, payment methods, theatre and screen usage, unpaid customers, theatres without shows, and total booking amounts.

Normalization through 1NF, 2NF, and 3NF reduces repetition and supports clear functional dependencies. Separating customer, booking, theatre, movie, screen, seat, payment, and booking-detail information also helps prevent insertion, update, deletion, and redundancy anomalies. Overall, the project provides a structured, consistent, and maintainable database model for cinema ticket-booking operations.

12. FUTURE ENHANCEMENTS

The following enhancements are proposed extensions consistent with the stated project objectives and non-functional requirements. They are not presented as completed features of the supplied implementation.

- Live seat availability and temporary seat locking during the booking transaction.
- Integrated online payment gateway support with payment confirmation and refund status.
- Web and mobile interfaces connected to the relational database for customer self-service.
- Automated SMS or email notifications for booking confirmation, cancellation, and show reminders.
- Role-based access for customers, booking staff, theatre administrators, and database administrators.
- Audit logging, automated backups, and recovery procedures for booking and payment records.
- Analytics dashboards for show occupancy, theatre usage, revenue, and booking trends.
- Stored procedures, triggers, and validation rules to strengthen booking integrity and reduce duplicate seat reservations.

13. REFERENCES

- Supplied PowerPoint: “Cinema Ticket Booking System BATCH-15 TT-3 121”, Review-2, 01-10-2026.
- Supplied cinema database screenshots containing DDL, sample data, normalization illustrations, and SQL query outputs.
- Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.
- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.



APPENDIX — PROJECT TEAM

- M.Tejaswaoop — 25B11CS563
- Ch.Dilip — 25B11CS166
- M.Siddharth Raj — 25B11CS617
- M.Santosh Naik — 25B11CS614

Project Guide: K. Vydehi Madam, Department of Computer Science and Engineering, Aditya University.

